

Übung 3: Systemarchitektur Ihres Projektes modellieren

Team 6 (Thema E-Vote) - Abgabe:

Link: https://github.com/jacque-schu/MSE_eVote

Gemeinsam als Team haben wir uns über Discord zum Thema „Systemarchitektur Ihres Projektes modellieren“ ausgetauscht. Für die erste folgende Aufgabe „Event Storming durchführen“ sowie für die Planung der weiteren Aufgaben haben wir in einem Miro-Board gemeinsam unsere Antworten zusammengetragen.

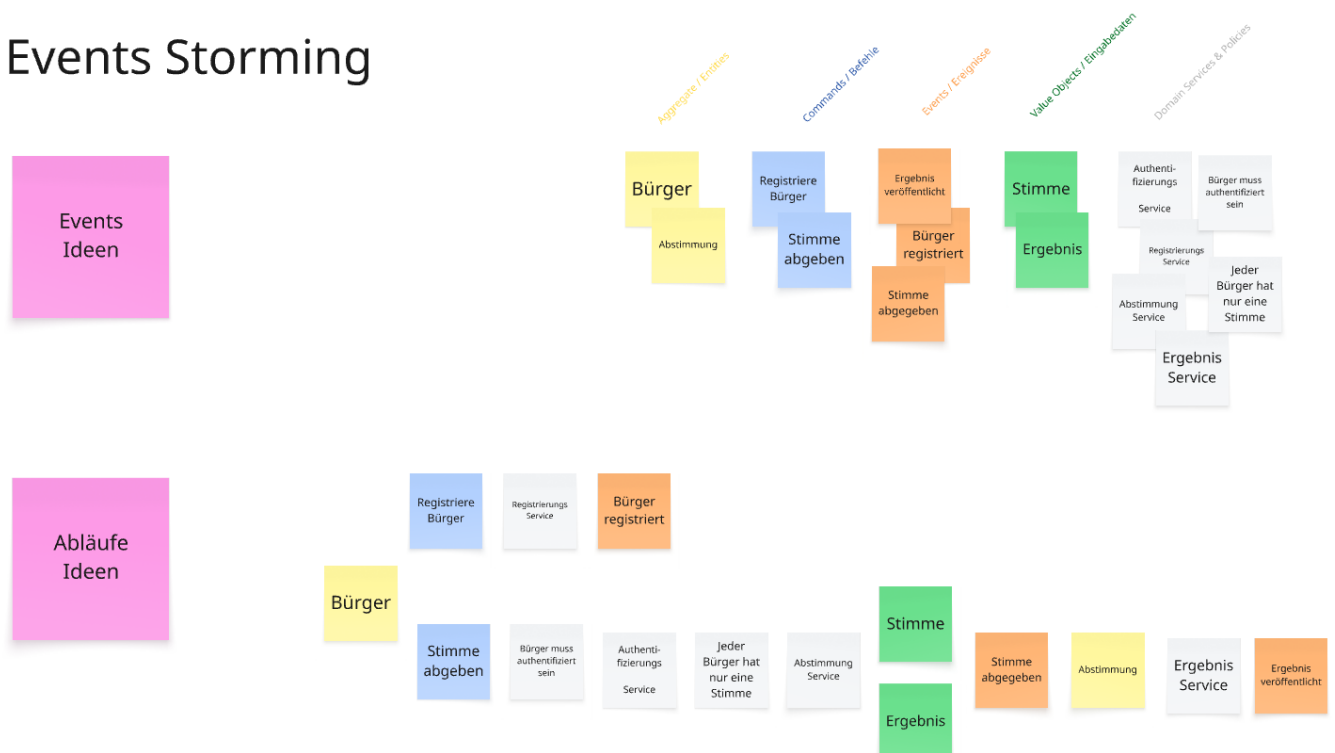
Link zum Miro-Board:

https://miro.com/welcomeonboard/UVpJaWtVUUhUeWJxSDNpekhBQXc0ZFFjaHJ6OW9uV2RLSHVWOWRIMjZhMnZyTkQwaWR0RytKalc0WmxSek3dFZoR1dWMHhNbFdEYks4YUJJcUJ1aG9QSzJDSnFNU05tQkR5N0kwQWUwSlh1RTThLVVFyS08vQ3BEODJiZk92bjZ3VHhHVHd5UWtSM1BidUtUYmxycDRnPT0hdjE=?share_link_id=303563442178

Lösungen aus unserem Miro-Board:

Aufgabe 1: Event Storming durchführen

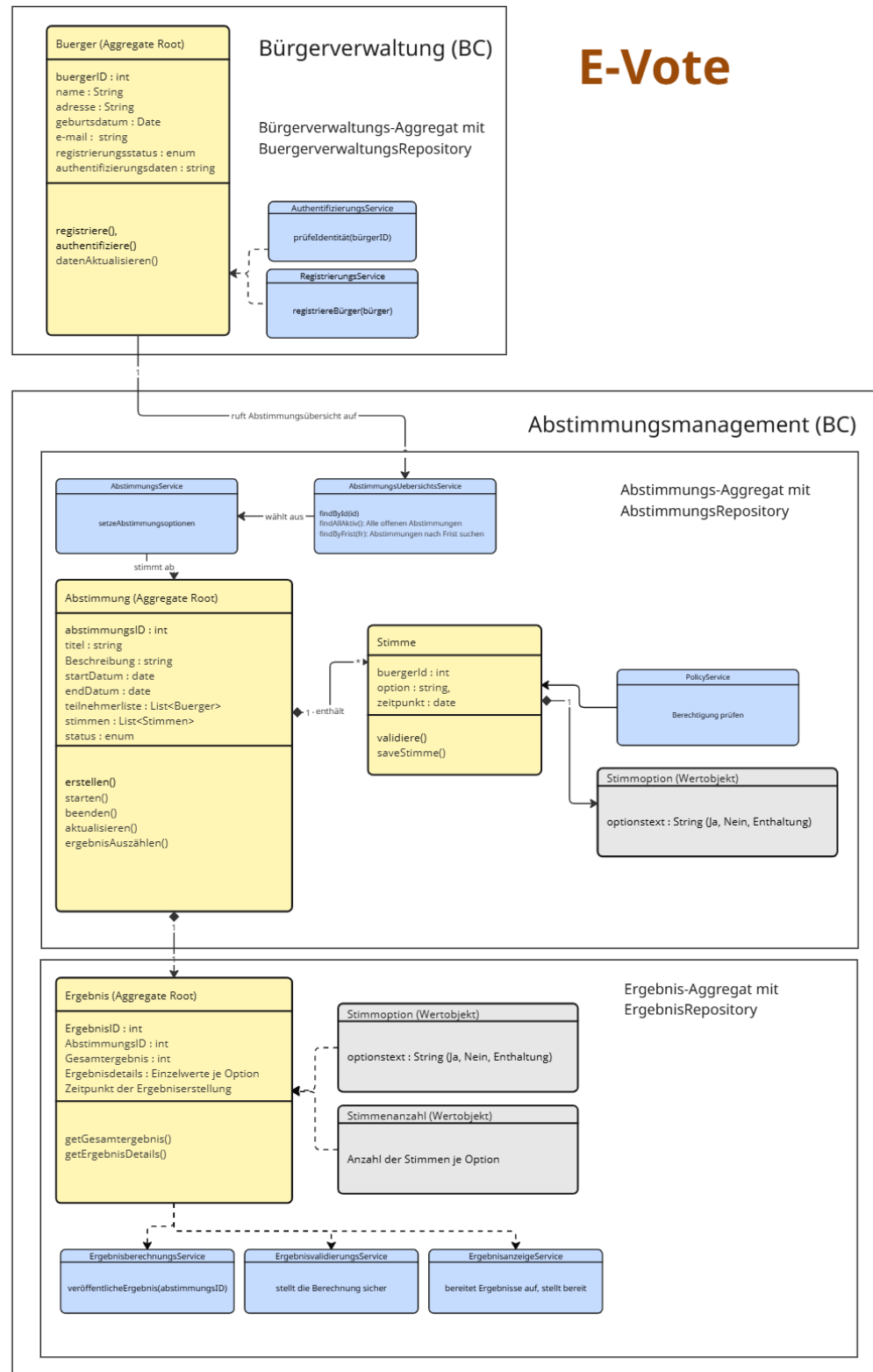
Events Storming



Aufgabe 2: Domänenmodell erstellen,
Aufgabe 3: Bounded Contexts identifizieren,
Aufgabe 4: Entitäten und Aggregates definieren,
und Aufgabe 5: Domain Services und Repositories zusammen:

Klassen-
diagramm
nach
DDD

E-Vote



zu **Aufgabe 4: Entitäten und Aggregates definieren:**

Bürgerverwaltung (Aggregate Root)

Entität: Bürger

Aggregate: Bürger (enthält alle Daten des Bürgers zur Registrierung, Aktivierung, E-Mail-Bestätigung, Statuswechsel)

Attribute: Bürger-ID, Name, E-Mail, Passwort(-Hash), Status (aktiv/gesperrt), Datum der Registrierung, E-Mail-Bestätigung (als Value Object), ggf. Adressdaten

Methoden: registrieren(), aktivieren(), sperren(), bestätigung_versenden(), bestätigung_validieren()

Abstimmungsmanagement (Aggregate Root)

Entität: Abstimmung

Aggregate: Abstimmung (enthält alle Daten zu einer Abstimmung, inkl. Frist und Status)

Attribute: Abstimmungs-ID, Titel, Beschreibung, Frist (Value Object: Start-/Enddatum), Status (offen/geschlossen/archiviert)

Methoden: abstimmung_erstellen(), abstimmung_schließen(), frist_prüfen(), beschreibung_aktualisieren()

(Teilnahme) / Stimme (Aggregate Root)

Entität: Stimme

Aggregate: Stimme (fasst die Stimmabgabe eines Bürgers für eine Abstimmung zusammen)

Attribute: Stimmen-ID, Bürger-ID (Referenz), Abstimmungs-ID (Referenz), Auswahl/Option, Zeitstempel

Methoden: abgeben(), ändern(), widerrufen()

Abstimmungsübersicht / Abstimmungsübersicht / Ergebnis-Aggregat (Aggregate Root)

Entität: Abstimmungsübersicht

Aggregate: Abstimmungsübersicht (bzw. Ergebnis)

Attribute: Liste der aktuellen Abstimmungen (Titel, Beschreibung, Frist, Status), ggf. Ergebnisse pro Abstimmung (Anzahl Stimmen je Option)

Methoden: abstimmungsuebersicht_anzeigen(), ergebnis_berechnen(), filtern(), sortieren()

zu 5. Domain Services und Repositories:

Domain Services:

1. **Authentifizierungsservice:** Authentifizierung und Token-Management, Validierung von Credentials
2. **Votingservice:** Verwaltung der Stimmabgabe, Berechtigungsprüfung, Ergebnisermittlung
3. **Fristenservice:** Prüfung und Verwaltung von Abstimmungsfristen

Repositories:

1. **BürgerverwaltungsRepository:** Persistenz und Laden von Nutzerkonten/Aggregaten
2. **AbstimmungsRepository:** Persistenz von Abstimmungs-Entitäten
3. **ErgebnisRepository:** Speicherung und Abfrage von Stimmen/Aggregaten

Übersicht der Repositories und Methoden:

BürgerverwaltungsRepository:

save(nutzer): Nutzer anlegen/aktualisieren
findById(id): Nutzerkonto laden
findByEmail(email): Nutzer über E-Mail suchen

AbstimmungRepository:

save(abstimmung): Abstimmung anlegen/aktualisieren
findById(id): Einzelne Abstimmung laden
findAllAktiv(): Alle offenen Abstimmungen
findByFrist(fr): Abstimmungen nach Frist suchen

ErgebnisRepository:

save(stimme): Stimme speichern
findByAbstimmungId(id): Stimmen für eine Abstimmung
findByNutzerId(id): Stimmen eines Nutzers finden

Begründung:

Services kapseln fachliche Operationen auf Entities; Repositories trennen Persistenz von Logik.

zu **6. Implementierungsstrategie:**

Zum aktuellen Zeitpunkt (Stand 03.11.2025) sind wir uns noch uneinig in der Entscheidung, welches Framework wir nutzen. Da unser Projekt mit Python umgesetzt wird, stehen in der näheren Auswahl Flask und FastAPI. Gemeinsam als Gruppe wiegen wir aktuell Vor- und Nachteile ab und werden die für uns passendere Auswahl treffen.

Durch diese offene Frage lassen wir aktuell Punkt 6. Implementierungsstrategie soweit offen und reichen diesen schnellstmöglich nach.

Resultat:

Nach mehreren Treffen als Gruppe und gemeinsamer Arbeit und Austausch am Projekt zu dieser Übung sind wir nun einen großen Schritt weiter. Wir können nun unsere bisher hier zusammengefassten Ausarbeitungen aus dem Miro-Board als nächstes für die Implementierung nutzen.

Uns ist auch bewusst, dass sich bestimmte Funktionalitäten in der Umsetzung sicher von unserer hier dargestellten Planung unterscheiden werden, wir werden diese Diagramme dementsprechend überarbeiten.